

AI Model for Agent Selection and Scoring Agent Generation

Connecting Text and Graph

Contexte

Prompt

Beth bakes 4, 2 dozen batches of cookies in a week.
If these cookies are shared amongst 16 people
equally, how many cookies does each person consume?

Ebiose Architect

?

Agent

Interpreter

Algebra

Analysis

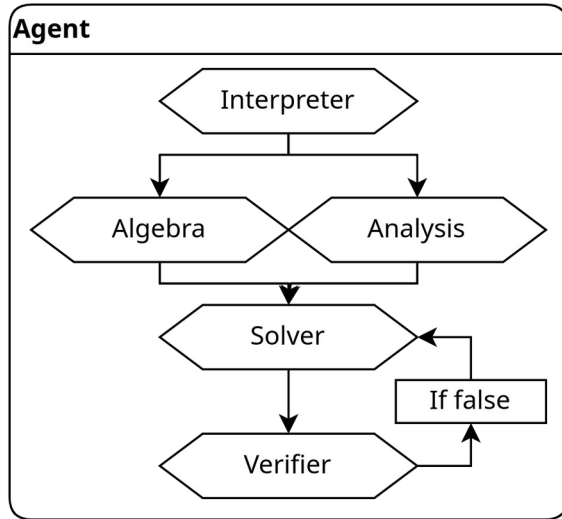
Solver

Verifier

If false

1. Comment sélectionner un agent pour répondre à un prompt en production ?
2. Comment scorer un agent lors de la génération d'agents avant de le tester ?

Pourquoi scorer un agent avant de le tester ?

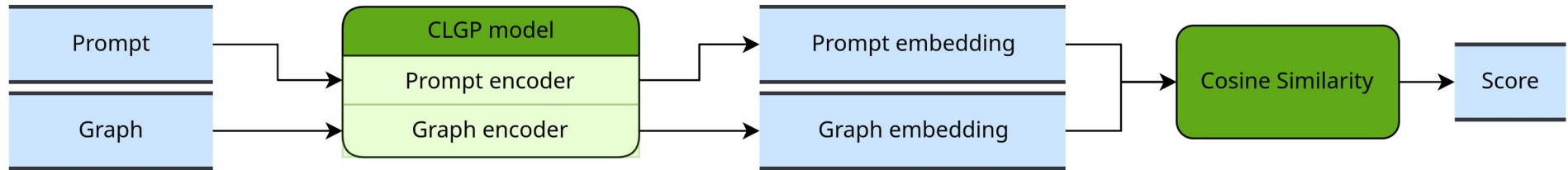


- Chaque nœud = 1 model
- ChatGPT 3.5 ~ 175 milliards de paramètres

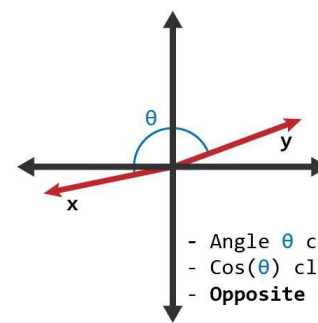
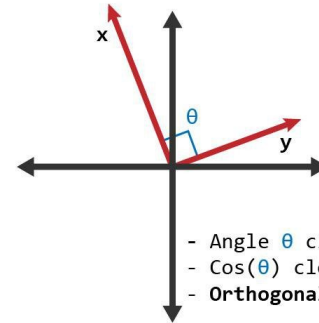
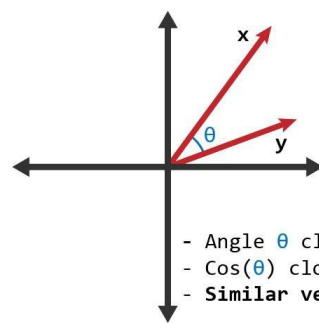
Avoir un model plus petit pour mesurer la capacité d'un agent à résoudre un problème :

Economie de ressources lors de la génération d'agent

Contrastive Language Graph Pair model

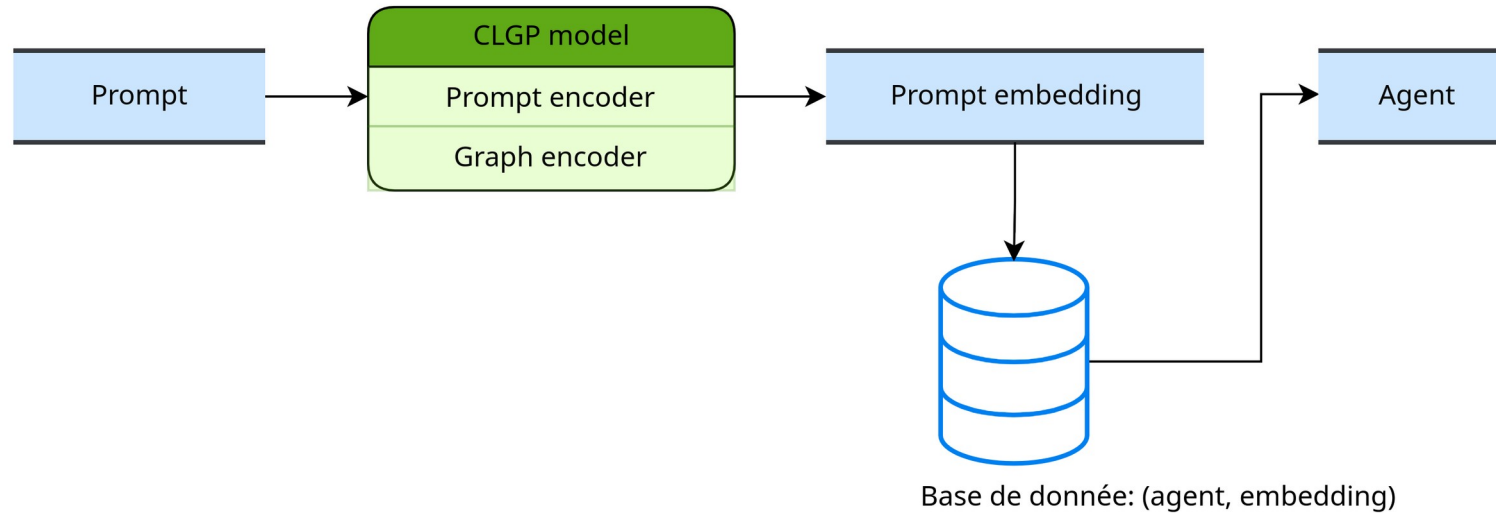


Cosine similarity :

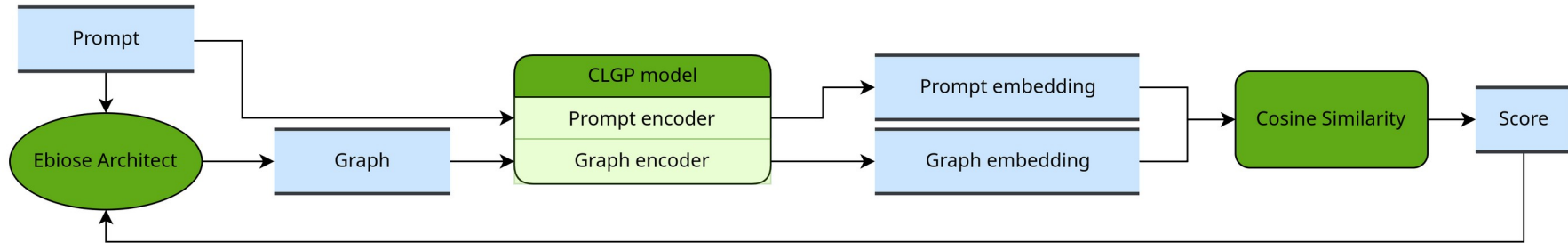


CLGP usage

1. Sélection d'agent dans une BDD :

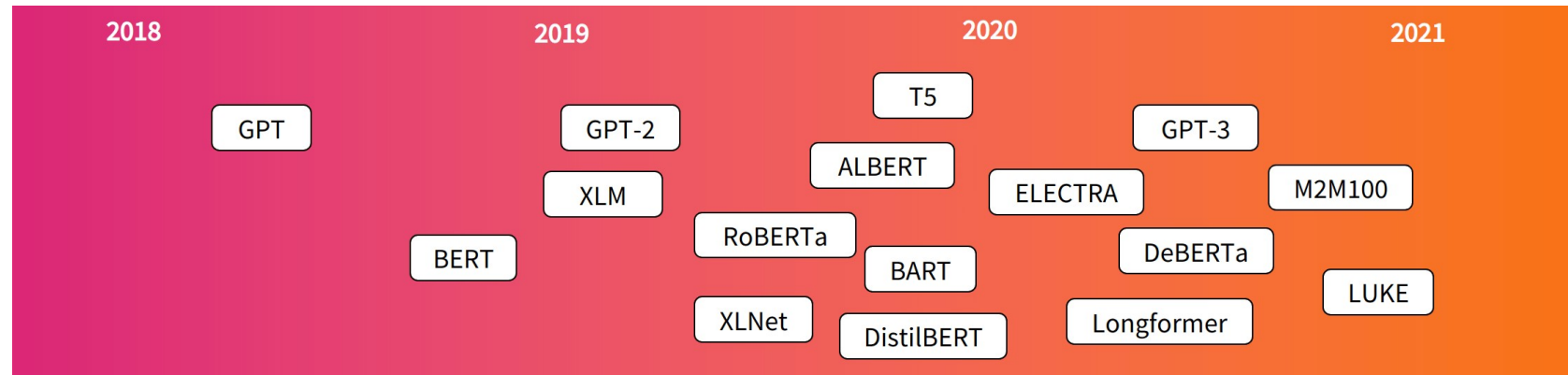


2. Scorer un agent lors de la génération d'agents :



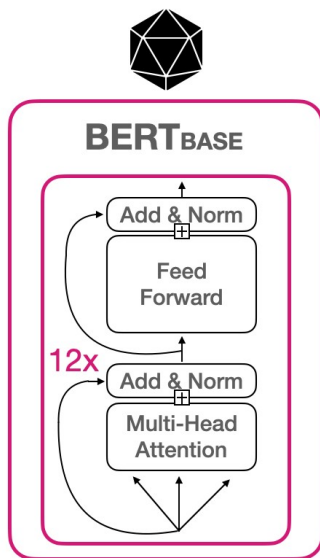
CLGP : prompt encoder

Chronologie des sorties des modèles populaires de Transformer :

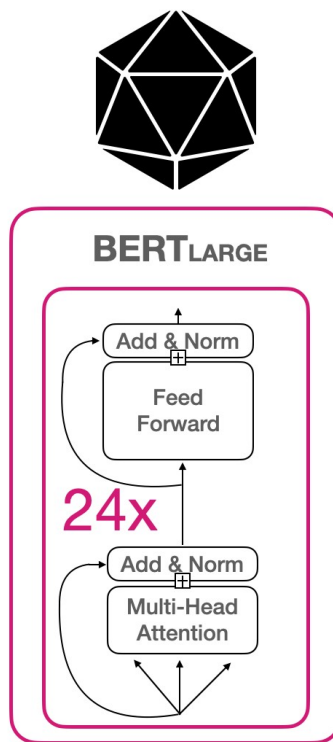


CLGP : prompt encoder

BERT Size & Architecture



110M Parameters

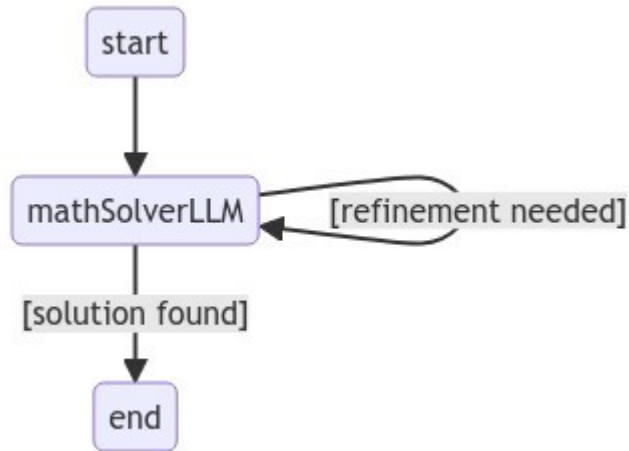


340M Parameters



CLGP : Graph encoder

Graph data representation :



Feature Matrix :

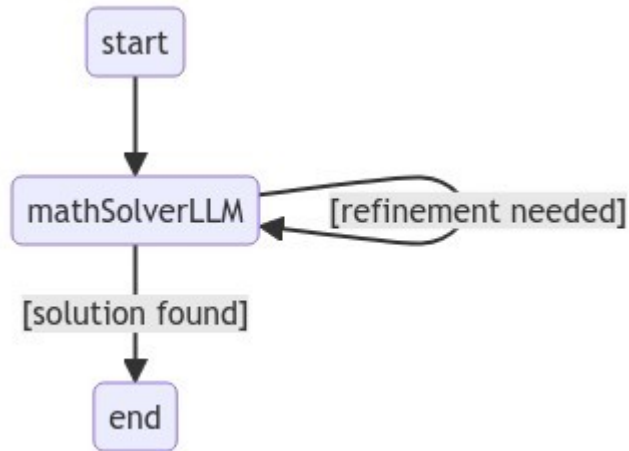
$$\mathbf{X} = \begin{pmatrix} \text{node feature}_1 \\ \text{node feature}_2 \\ \text{node feature}_3 \\ \text{node feature}_4 \\ \text{node feature}_5 \end{pmatrix}$$

Adjacency Matrix :

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

CLGP : Graph encoder

Graph data representation :



Node feature₂ = {'id': 'mathSolverLLM',

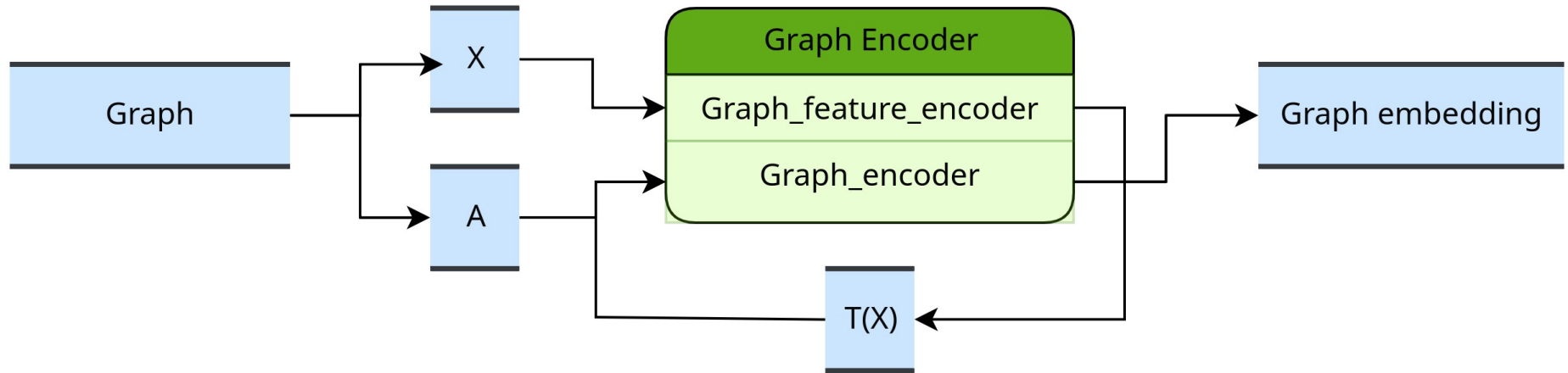
'name': 'mathSolverLLM',

'purpose': 'To both deeply and methodically analyze and solve the provided mathematical issues, iterating as necessary to achieve a definitive solution.',

'type': 'LLM',

'prompt': "Meticulously analyze the mathematical problem at hand. Begin with a sequential logical breakdown, exploring possible solutions methodically. Depending on your findings, choose to: 1. Announce a complete solution with the addition '[solution found]' if the problem is fully resolved. 2. Signal the need for further investigation or clarification with '[refinement needed]', articulating any specific areas that require deeper analysis or additional information.", 'model': 'azure-gpt-3.5-turbo'}

CLGP : Graph encoder

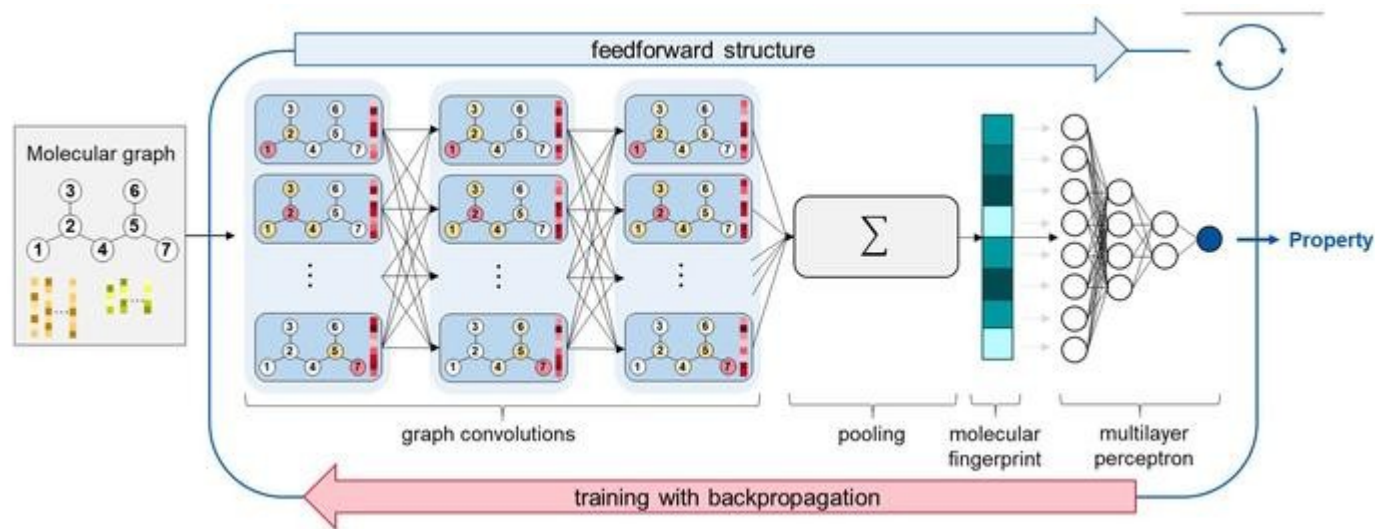


- Graph_feature_encoder = Transformer (BERT)
- Graph_encoder = Graph Convolutional Network (GCN)
ou Graph Attention Network (GNN)

CLGP : Graph encoder

Graph Convolutional Network (2017)

$$H^{(1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} X W^{(0)} \right)$$
$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right).$$



Physical Pooling Functions in Graph Neural Networks for Molecular Property Prediction by Artur Schweidtmann & Jan G. Rittig

CLGP : Graph encoder

Graph Attention Network (2017)

$$e_{ij} = a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j)$$

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}$$

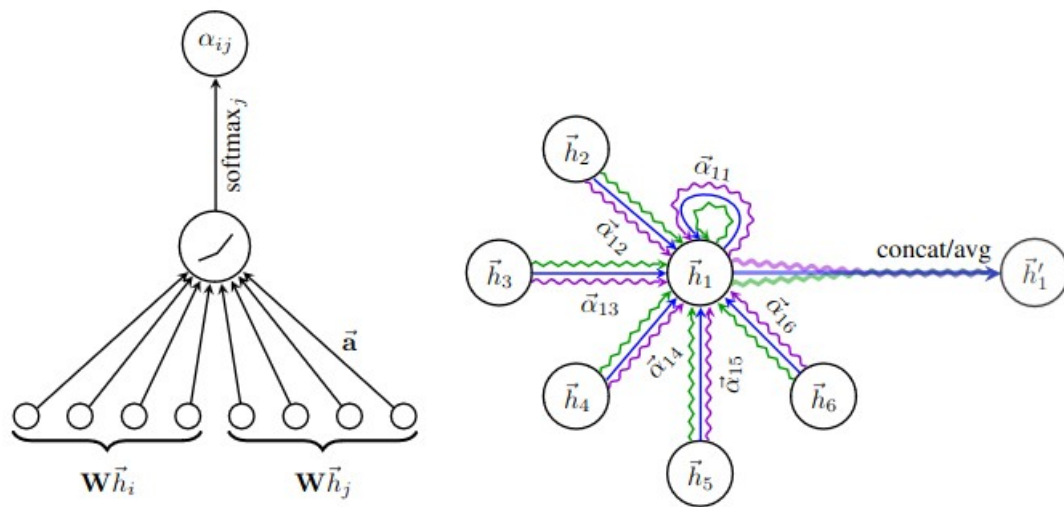
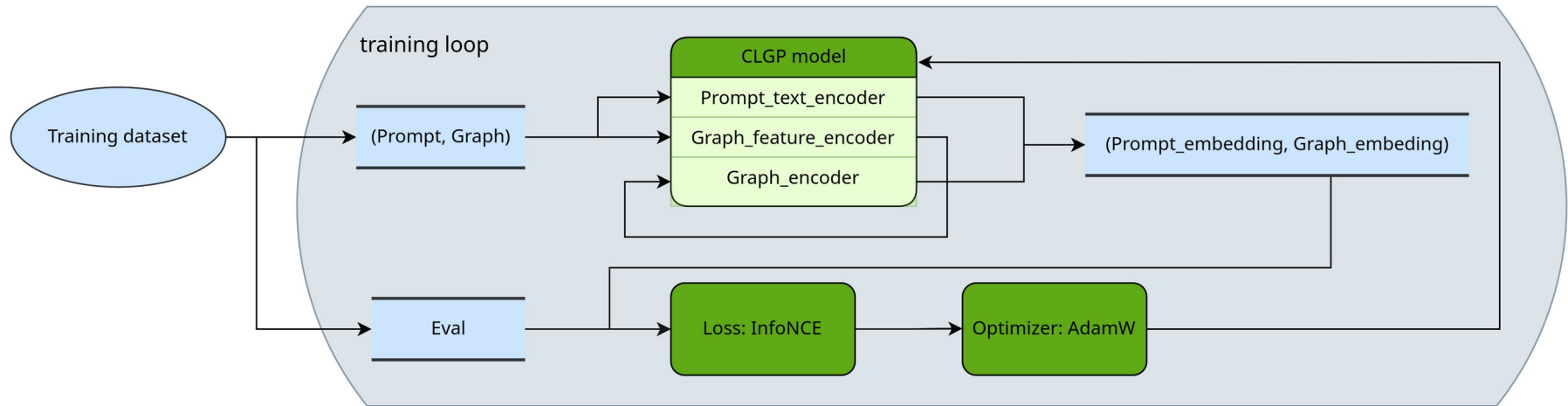
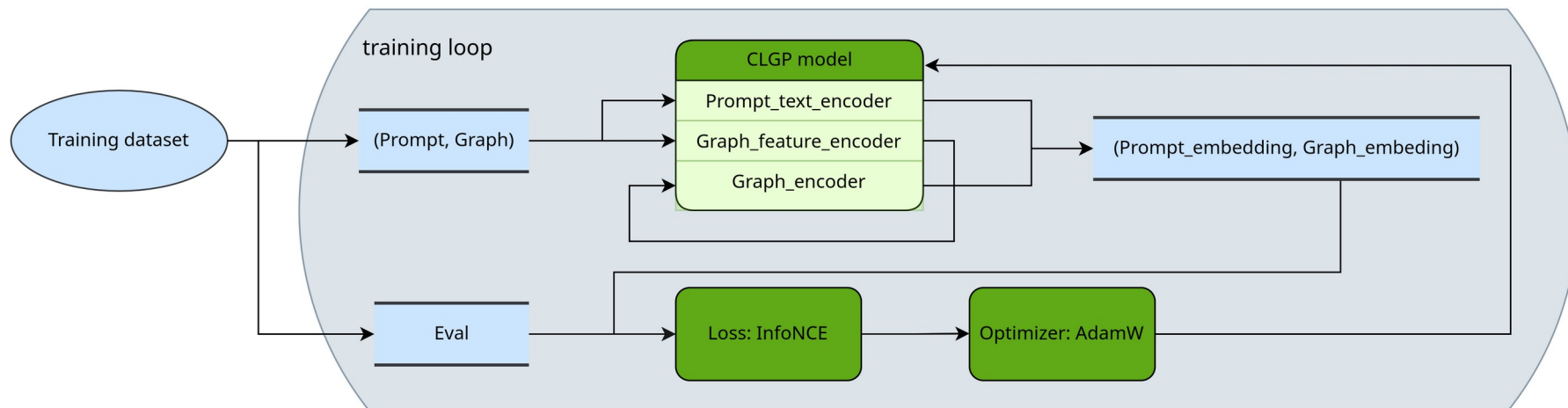


Figure 1: **Left:** The attention mechanism $a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j)$ employed by our model, parametrized by a weight vector $\vec{a} \in \mathbb{R}^{2F'}$, applying a LeakyReLU activation. **Right:** An illustration of multi-head attention (with $K = 3$ heads) by node 1 on its neighborhood. Different arrow styles and colors denote independent attention computations. The aggregated features from each head are concatenated or averaged to obtain $\vec{h}'_1$.

CLGP : Entraînement



CLGP : Entraînement



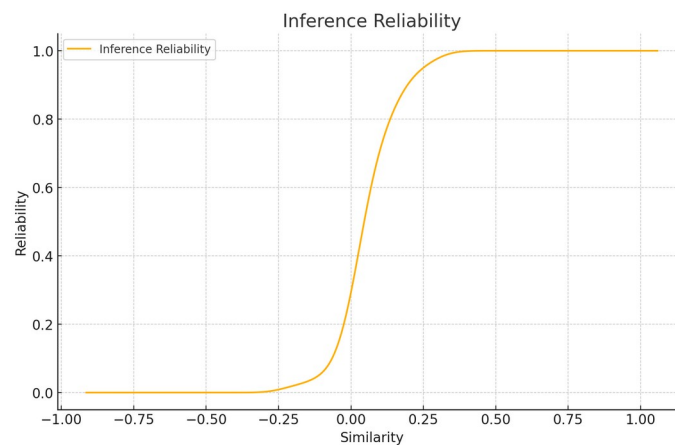
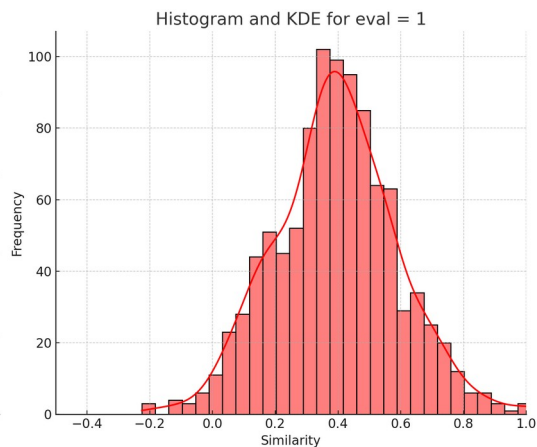
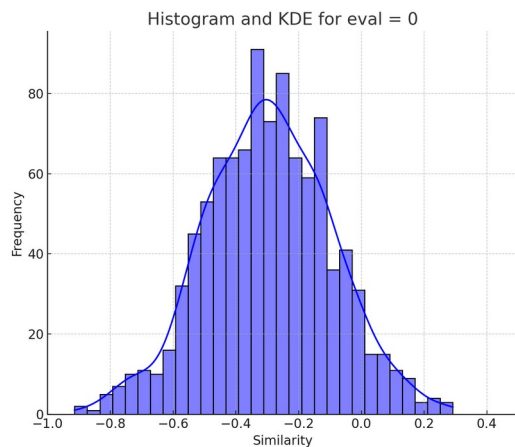
$$\text{loss} = \frac{1}{N} \sum_{i=1}^N \left[-y_i \log \left(\sigma \left(\frac{\hat{\mathbf{u}}_i \cdot \hat{\mathbf{v}}_i}{\text{température}} \right) \right) - (1 - y_i) \log \left(1 - \sigma \left(\frac{\hat{\mathbf{u}}_i \cdot \hat{\mathbf{v}}_i}{\text{température}} \right) \right) \right] \quad (1)$$

où $\hat{\mathbf{u}}_i$ et $\hat{\mathbf{v}}_i$ sont respectivement les embeddings normalisés du graphe et du prompt, y_i est l'évaluation, $\sigma(\cdot)$ est la fonction sigmoïde, et N est la taille du batch.

CLGP : Evaluation

$$\text{Test datasets} = \left\{ \begin{array}{l} \{(\text{prompt}, \text{graph}, \text{eval}) \mid \text{prompt} \in (\text{Train} \cup \text{Val}) \wedge \text{graph} \in (\text{Train} \cup \text{Val})\} \\ \{(\text{prompt}, \text{graph}, \text{eval}) \mid \text{prompt} \in \text{Train} \cup \text{Val} \wedge \text{graph} \in \text{Train} \cup \text{Val}\} \\ \{(\text{prompt}, \text{graph}, \text{eval}) \mid \text{prompt} \in \text{Train} \cup \text{Val} \wedge \text{graph} \in \text{Train} \cup \text{Val}\} \end{array} \right.$$

Pour chaque dataset de test :



CLGP : Implémentation

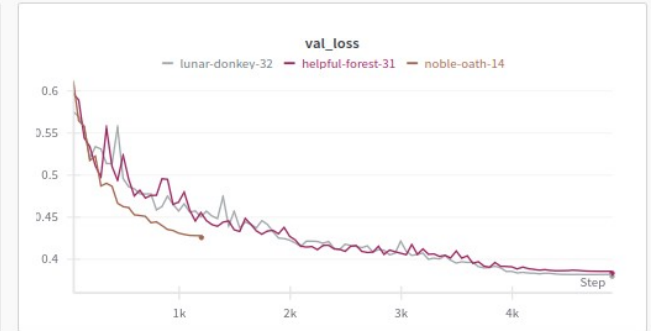
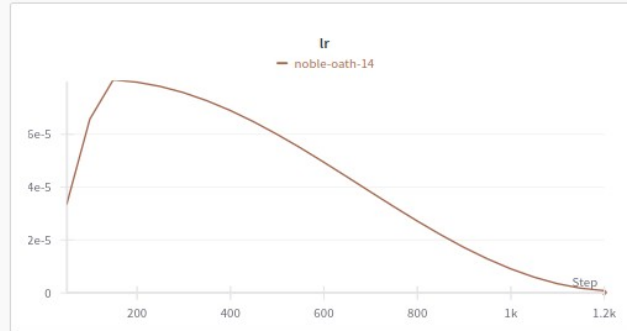
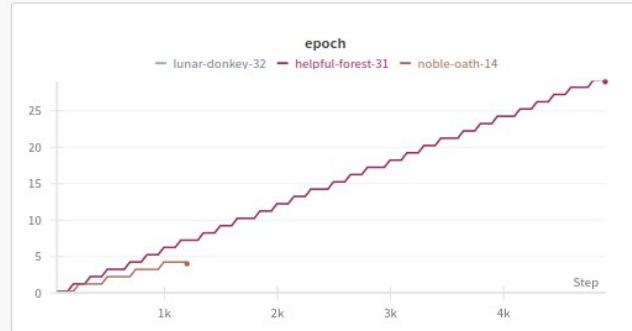
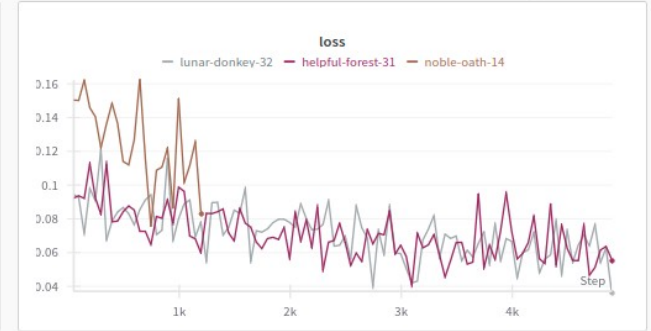
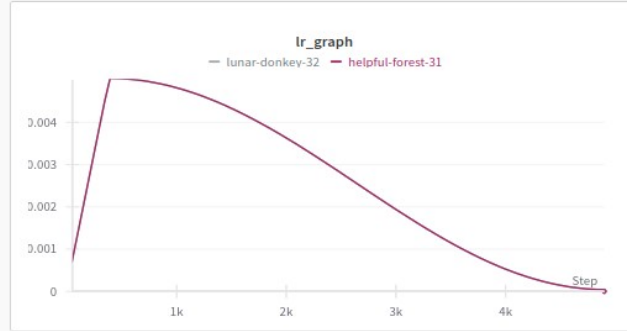
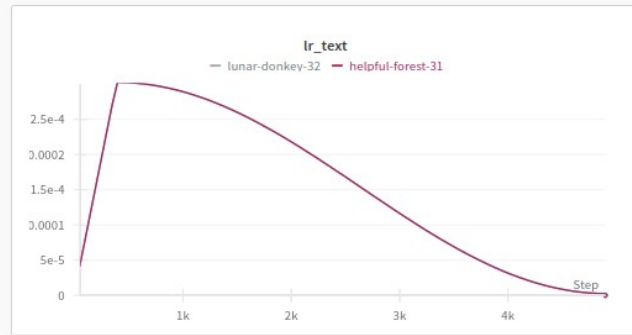


Training runs :

Google Colaboratory

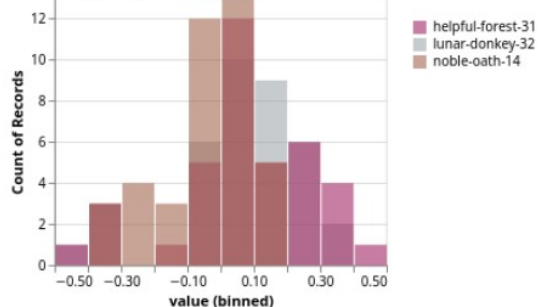


Résultats des premiers entraînements

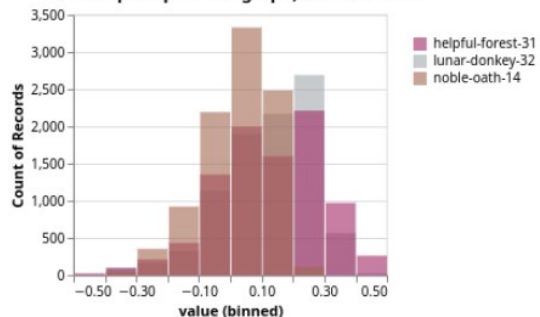


Résultats des premiers entraînements

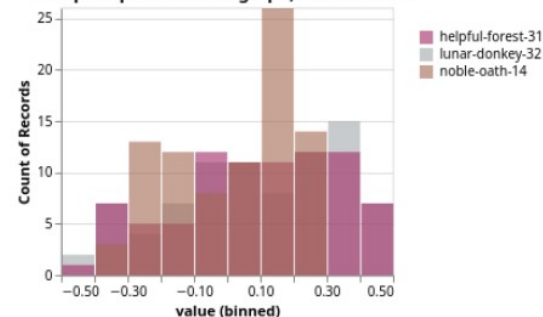
test prompt & test graph, evaluation = 1



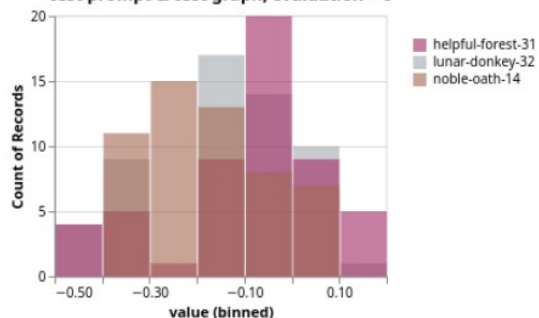
trained prompt & test graph, evaluation = 1



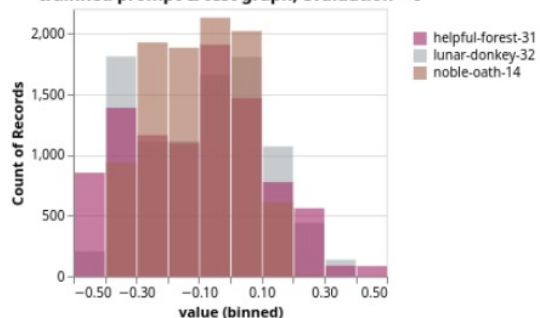
test prompt & trained graph, evaluation = 1



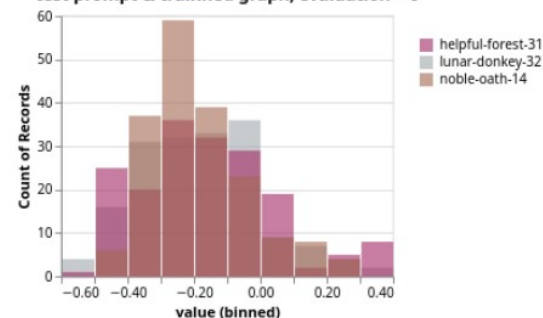
test prompt & test graph, evaluation = 0



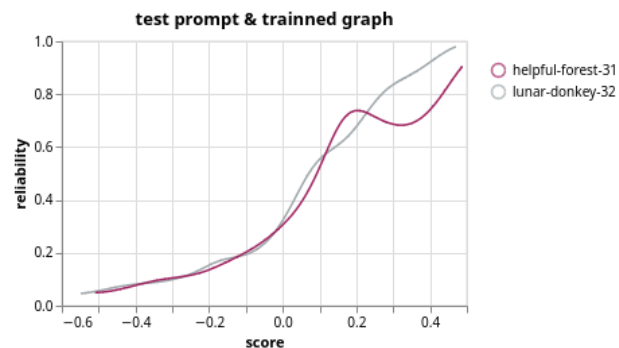
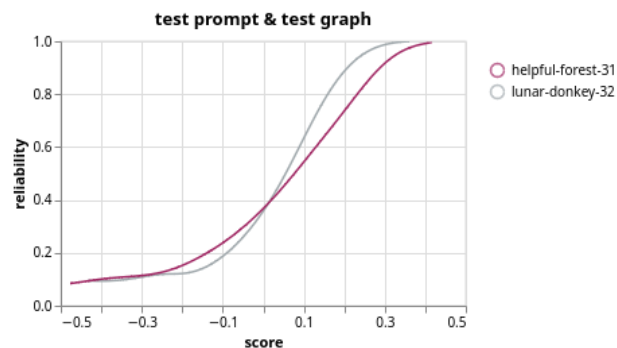
trained prompt & test graph, evaluation = 0



test prompt & trained graph, evaluation = 0



Résultats des premiers entraînements



Pistes d'améliorations

- Expérimenter d'autres architectures de graph encodeurs
- Expérimenter d'autres architectures de text encoder pré-entraînés (tester fine tuning LoRA)
- Explorer l'espace des hyper-paramètres: nombre de layers au GNN, learning rates, dropout rates, nombre d'epochs...
- Faire de la data augmentation
- Faire du feature engineering sur les node_features

Conclusion

